

线性关系与存储表示

同一逻辑序列怎样换表示，操作为什么随之改变

408 · 数据结构 Topic DS-01

母问题：逻辑线性关系怎样被有限存储兑现？

线性表不是“数组或链表的名字”，而是一组对元素顺序和操作结果的契约。同一个逻辑表及其操作契约可以由不同物理表示实现：

同一(Linear Relation, Operation Contract) 允许选择不同 Representation

分析时先固定逻辑对象与操作语义，再检查某种表示需要维护哪些字段和不变量，最后比较定位、移动、指针跳转、额外空间与局部性的成本。不同表示必须给出相同的抽象结果，但不必承担相同的实现代价。

本册定位与 Owner 边界

- **Owns**：线性表 ADT、顺序表、单链表、双链表、静态链式表示；数组的多维地址映射与特殊矩阵压缩；插入/删除/按位访问/按值查找的状态变化与边界。
- **Uses**：Atlas Foundation 的问题规模、基本操作、最坏/平均/摊还成本和成本向量语言。
- **Outputs**：向 DS02 提供“底层顺序/链接表示”，向 DS09 提供“有序序列上的访问与插入代价”；栈、队列、字符串和索引只调用本册表示，不复制本册机制。
- **Stops**：受限端点语义归 DS02；KMP 归 DS03；树/图节点结构归 DS04/DS07；BST、AVL、B/B+ 的有序索引机制归 DS09。

目录

1	先把“表”与“存储”分开	3
1.1	三层词典：逻辑对象、存储方法与具体表示不要混名	3
2	顺序表：地址映射换取按位访问	3
2.1	长度、容量与合法状态	4
2.2	插入与删除是区域移动	4
3	多维数组与特殊矩阵：坐标如何映射为线性偏移	4
3.1	压缩存储：先找独立元素，再建立可逆编号	4
4	链式表示：指针关系换取局部修改	5
4.1	链表的不变量	5
4.2	局部插入的完整生命周期	5
4.3	单链表逆置：真正的不变量是“未处理后缀不能失联”	5
4.4	线性表 ADT 的四条逻辑约束	6
4.5	头指针、头结点、首元结点与空表	6

4.6	四种链式表示：同一关系，不同边界责任	6
4.7	单链表的两种建表顺序	7
4.8	双链表插入与删除的四条边	7
4.9	链表题的通用位置算法：先把位置关系变成状态	7
4.10	数组、矩阵与压缩存储的完整口径	8
4.11	循环链表：只改一条边，所有终止条件都要重写	8
4.12	静态链表：把地址换成游标，同时管理两条链	9
4.13	链表算法的可证明模板	9
5	同一操作的两套成本向量	9
6	生成性例子：把“按值删除”拆成两个阶段	10
7	代码把模型变成可执行状态转移	10
7.1	顺序表：先保证容量，再从后向前移动	10
7.2	单链表：局部 $O(1)$ 从“已知前驱”开始	11
7.3	按值删除：Locate 与 Update 在代码中也必须分层	12
7.4	单链表逆置：代码必须先保存旧后继	13
7.5	不变量检查与测试不是生产操作的成本	13
8	空、首、尾与重复值不是补丁	14
9	概念边界：把相似说法还原成判据	15
10	压缩复原	15
11	全科坐标与跨册交接	15
12	来源处理	15

1 先把“表”与“存储”分开

抽象线性表 $L = (a_1, a_2, \dots, a_n)$ 只承诺元素的线性次序、长度和允许操作。它不承诺元素在内存中连续，也不承诺一次操作一定只访问一个地址。若把逻辑位置记为 i ，必须先区分：

层次	要回答的问题	不能直接推出的结论
逻辑关系	谁是第 i 个元素？插入后次序怎样？	元素一定相邻
操作契约	按位访问、按值查找、插入、删除的输入是什么？	每种操作都同样便宜
物理表示	数据、长度、容量、指针和空闲单元怎样保存？	只看元素字段就能判断状态
成本模型	计数对象是比较、移动、指针跳转还是 I/O？	一个 $O(\cdot)$ 覆盖所有场景

1.1 三层词典：逻辑对象、存储方法与具体表示不要混名

408 里最容易发生的概念滑移，是把“线性表、数组、顺序表、链表”放在同一层比较。更稳的分层是：

层次	核心问题	典型名称
逻辑关系与操作契约	元素之间是什么关系？允许怎样访问、插入和删除？	线性表、有序表、栈、队列
存储方法	元素及其关系靠什么机制编码？	顺序存储、链式存储、索引、散列
具体表示	真实维护哪些字段、边界和不变量？	顺序表、单链表、双链表、循环链表

因此“数组就是物理结构、树图就是逻辑结构”只能作非常粗的入门类比，不能作为稳定术语。数组在程序语言里本身也是一种具体容器/数据结构；在线性表语境中，它通常承担连续、可下标寻址的存储载体。链表则是用链接域显式保存邻接关系的一类具体表示。真正属于存储方法层的词，是**顺序存储**、**链式存储**、**索引和散列**。

这也解释了“数据运算到底算不算数据结构的一部分”的表面冲突：408 教材通常把**逻辑结构**、**存储结构**、**基本运算**作为课程研究的三块核心内容；ADT 视角则把抽象状态和操作语义作为公开契约，把具体存储字段与实现算法隐藏在表示内部。两种口径是在不同抽象层说同一件事，不应写成彼此否定。

最小反例：同一个“插入第 k 个”

对逻辑表 $[a, b, c, d]$ 在位置 3 插入 x ，抽象结果只有一个： $[a, b, x, c, d]$ 。顺序表需要把 c, d 向后移动；链表只需让 x 接入正确的指针链，但前提是已经拿到插入点的前驱。若题目只给关键字而不给位置，寻找前驱的成本又回来了。

2 顺序表：地址映射换取按位访问

顺序表把元素放入一段可按下标解释的连续区域。设首地址为 B ，每个元素占 w 个存储单元，则零基下标 i 的地址为

$$\text{addr}(A[i]) = B + iw.$$

这条式子不是“数组 API 口诀”，而是顺序表能在 $\Theta(1)$ 时间定位第 i 个元素的原因：位置到地址是一次算术映射，不需要沿中间元素搜索。

2.1 长度、容量与合法状态

动态顺序表至少维护 $(data, length, capacity)$ 。合法状态包括

$$0 \leq length \leq capacity, \quad 0 \leq i < length \Rightarrow A[i] \text{ 是有效元素.}$$

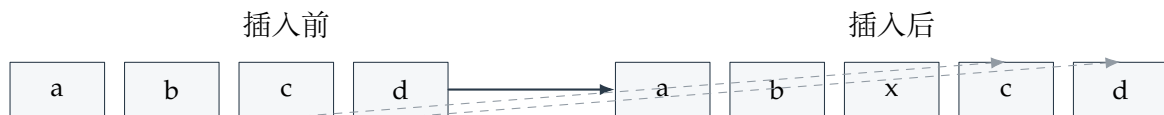
插入前先检查 $length < capacity$ ；若需要扩容，旧元素必须按原次序复制到新区域，再改变容量和写入位置。扩容后的单次复制可能是 $\Theta(n)$ ，但若容量按固定倍数增长，连续追加的总复制量可摊还到每次 $O(1)$ ；这里的“摊还”不是把一次最坏成本改写成 $O(1)$ 。

2.2 插入与删除是区域移动

在长度为 n 的表中，在逻辑位置 k ($0 \leq k \leq n$) 插入：

$$\text{检查容量} \rightarrow A[n-1] \rightsquigarrow A[k] \text{ 右移} \rightarrow A[k] \leftarrow x \rightarrow length++.$$

需要移动 $n-k$ 个元素；在头部插入为 $\Theta(n)$ ，尾部追加在有剩余容量时为 $\Theta(1)$ 。删除位置 k 则把区间 $A[k+1 \dots n-1]$ 左移，移动 $n-k-1$ 个元素，再递减长度。覆盖或清理被删除槽位是实现选择，但不能让无效槽位继续被逻辑遍历访问。



3 多维数组与特殊矩阵：坐标如何映射为线性偏移

数组的维数是逻辑坐标的个数，内存地址仍是一维序列。行优先、列优先不是两种数组对象，而是两种线性化函数。设二维数组 $A[L_1..U_1][L_2..U_2]$ ，每个元素占 w 个存储单元， $N_1 = U_1 - L_1 + 1$ ， $N_2 = U_2 - L_2 + 1$ ，则

$$\text{addr}_{\text{row}}(A[i][j]) = B + ((i - L_1)N_2 + (j - L_2))w,$$

$$\text{addr}_{\text{col}}(A[i][j]) = B + ((j - L_2)N_1 + (i - L_1))w.$$

两式都从同一问题生成：目标坐标之前，线性序列中已经排了多少个元素？下界不从 0 或 1 开始时，必须先减去逻辑下界；只背 $in + j$ 会在非零下界题中错位。

对 d 维数组，行优先偏移可递推为

$$\text{offset} = (((i_1 - L_1)N_2 + (i_2 - L_2))N_3 + \dots)N_d + (i_d - L_d),$$

本质是混合进位制，不需要另背展开式。

3.1 压缩存储：先找独立元素，再建立可逆编号

特殊矩阵只有在未存部分可由结构规则恢复时才能压缩。先确定独立元素集合，再为它建立连续编号 $k = f(i, j)$ ：

- 对称矩阵只存一个三角区，因为 $a_{ij} = a_{ji}$ ；访问另一半时先交换坐标再编号。
- 上/下三角矩阵存一个三角区和一个公共常数；另一三角区的元素不占独立槽位。
- 对角、带状矩阵只存满足带宽约束的坐标；带外元素由题设常数恢复。

- 稀疏矩阵通常保存非零三元组 $(row, column, value)$ ；它换掉了连续随机定位能力，查找需额外搜索或索引。

例如按行存储下三角矩阵（含主对角线），零基坐标满足 $0 \leq j \leq i < n$ 。第 i 行之前已有 $i(i+1)/2$ 个元素，所以

$$k = \frac{i(i+1)}{2} + j.$$

公式证据是“前面完整行的长度和 + 当前行内偏移”。若改用一基下标或上三角按列存储，应重新数前置元素，不能机械平移。

数组映射题的停止条件

先写合法坐标域、存储顺序和独立元素集合；再数目标元素之前的槽位，最后乘元素宽度并加首地址。编号必须落在 $[0, m-1]$ ，其中 m 是实际槽数；若两个本应独立的坐标映到同一编号，映射就不合法。

4 链式表示：指针关系换取局部修改

单链表节点保存 $(value, next)$ ，表对象通常保存头指针和可选的尾指针、长度。逻辑相邻不等于物理相邻： a_i 的后继由指针给出，而不是由地址加 w 得到。双链表再保存 $prev$ ，换取从后继回到前驱的能力。

4.1 链表的不变量

对无环单链表，必须保持：

1. 头指针要么为空，要么指向第一个有效节点；
2. 从头沿 $next$ 访问，每个有效节点至多出现一次，最终到达空指针；
3. 若维护 $tail$ ，则 $tail$ 是最后节点且 $tail.next = null$ ；
4. 若维护 $length$ ，它等于从头可达节点数。

双链表还要保持相邻互逆： $u.next = v \Rightarrow v.prev = u$ 。任一方向只改一条边都会暂时破坏结构，后续操作可能沿错误方向丢失节点。

4.2 局部插入的完整生命周期

已知前驱 p ，在其后插入新节点 x 的最小动作是：

$$x.next \leftarrow p.next, \quad p.next \leftarrow x.$$

若 p 原来是尾节点，还要更新 $tail$ ；若向空表插入，还要同时更新 $head$ 。删除节点 x 时，必须先拿到前驱 p ，令 $p.next \leftarrow x.next$ ，再处理尾指针、长度和资源回收。“指针修改 $O(1)$ ”只描述已定位节点后的局部动作，不包括按关键字寻找节点。

4.3 单链表逆置：真正的不变量是“未处理后缀不能失联”

原地逆置最危险的不是写错新方向，而是在覆盖 $current.next$ 时丢掉它原来指向的后缀。稳定状态应同时维护三个角色：

$$previous = \text{已经逆置完成的前缀入口}, \quad current = \text{当前节点}, \quad next = \text{尚未处理后缀入口}$$

每轮必须先保存旧后继，再改边：

$$next \leftarrow current.next, \quad current.next \leftarrow previous, \quad previous \leftarrow current, \quad current \leftarrow next.$$

循环不变量是：*previous* 所指前缀已经完全反向，*current* 所指后缀仍保持原方向，而且后缀始终有入口。当 *current* = *null* 时，*previous* 成为新表头；若维护 *tail*，旧表头成为新表尾并满足 *tail.next* = *null*。

这个模型比“背四行代码”更可迁移：删除、链表重排、局部反转和双链表换边都遵守同一条安全原则——覆盖一条仍承担唯一可达性的旧边以前，先保存后续仍需访问的对象。

4.4 线性表 ADT 的四条逻辑约束

外部资料把线性表的定义拆成四条容易被题面隐藏的约束。设非空表为 $L = (a_1, a_2, \dots, a_n)$ ，则：

1. 每个元素都有唯一的直接前驱和直接后继，首元素没有直接前驱，尾元素没有直接后继；
2. 元素之间是单一的线性次序，而不是任意的多对多关系；
3. n 是有效元素个数，不等于分配槽位数，也不等于节点地址数量；
4. 插入、删除会改变后继关系和位置编号，查找只观察而不改变逻辑次序。

“线性”描述的是关系，不是内存布局。顺序表、带头结点的链表和静态链表都可以实现同一个 ADT；它们的差异来自怎样兑现位置、后继、空闲和更新契约。

4.5 头指针、头结点、首元结点与空表

链表题必须先声明是否采用头结点。头指针是保存链表入口的指针变量：带头结点时通常指向头结点；不带头结点时通常直接指向首元结点。头结点是位于首元结点之前的可选哨兵，不计入线性表的数据元素个数；它的数据域可以不用，也可以承载长度等辅助信息，但不能把它当成第一个有效数据元素。首元结点才是真正保存 a_1 的节点。

若采用带头结点表示：

$$head \neq null, \quad first = head.next,$$

空表的判据是 *head.next* = *null*，而不是 *head* = *null*。这样做的价值不是增加一个“没用节点”，而是把“在首元结点前插入”和“在普通前驱后插入”统一成同一种边操作。若题目没有头结点，头插、删除第一个数据节点和空表恢复必须单独分支；不能把带头结点代码的指针起点直接套过去。

4.6 四种链式表示：同一关系，不同边界责任

表示	新增的状态约束	典型收益与代价
单链表	节点沿 <i>next</i> 最终到 <i>null</i>	实现简单；从后继不能直接回前驱
双链表	$u.next = v \Leftrightarrow v.prev = u$	已知节点时双向删除/反向遍历方便；每节点多一个指针
循环链表	尾节点后继回到首节点或头结点	无 <i>null</i> 尾端，轮转和循环调度自然；终止条件必须改为“回到起点”
静态链表	指针字段改为数组下标游标，空闲链也必须可达	不依赖动态指针；容量固定，空闲回收与越界由数组负责

循环链表的关键不是“把最后一个 *next* 改一下”，而是同时改写所有遍历终止条件：从首元节点出发时，停止条件是再次遇到首元节点；若采用带头结点，空表仍可用 *head.next* = *head* 表示。带尾指针时，非空循环单链表满足 *tail.next* = *head*；因此尾插只需修改新节点、旧尾节点和 *tail* 三条关系。删除唯一数据节点后必须恢复这个闭环，否则下一次插入会沿旧节点或 *null* 走错。

静态链表用整数下标代替地址：节点数组保存 $(data, nextIndex)$ ， $nextIndex$ 指向下一个数组槽位，特殊值表示空。它实际上维护两条链：一条从表头可达的有效链，另一条从空闲表头可达的空闲链。插入先从空闲链摘一个槽位，再把它接入有效链；删除先把节点摘出有效链，再头插回空闲链。若只修改有效链而不回收空闲槽，结构在短题中看似正确，连续插入后却会错误报告“满表”。

4.7 单链表的两种建表顺序

头插法每次把新节点接到当前首元之前，因此输入序列 $a_1, a_2, \dots, a_n$ 建出的逻辑序列为 $a_n, \dots, a_2, a_1$ ；尾插法保持输入次序，但需要维护尾指针或每次走到尾部。两种方法的差异可以由同一状态转移解释：

头插： $x.next \leftarrow head.next, head.next \leftarrow x$;

尾插： $tail.next \leftarrow x, x.next \leftarrow null, tail \leftarrow x$.

如果题目要求“按输入顺序建表”，头插后还必须反转，或直接选尾插；不能只看每次插入都是 $O(1)$ 就忽略最终关系。

4.8 双链表插入与删除的四条边

在双链表中，已知相邻节点 p 与 $q = p.next$ ，把 x 插入二者之间必须完成：

$$x.prev \leftarrow p, \quad x.next \leftarrow q, \quad p.next \leftarrow x, \quad q.prev \leftarrow x.$$

如果 q 为空，第四条赋值改为更新 $tail$ ；如果 p 是头结点，仍应保留头结点的边界约定。删除 x 时则令 $x.prev.next = x.next$ （若存在前驱）并令 $x.next.prev = x.prev$ （若存在后继），再修复 $head/tail/length$ 。四步的重点不是固定代码顺序，而是保证在任何中间时刻仍保留至少一条可恢复的旧链；不能先丢掉唯一的后继引用。

4.9 链表题的通用位置算法：先把位置关系变成状态

外部真题与拓展题反复出现的“倒数第 k 个、共同后缀、重排、判环、回文”并不是新的链表结构，而是对节点关系增加了一个位置约束。可统一为：

1. **快慢指针**：让两个指针的速度差编码目标距离。例如先让 $fast$ 比 $slow$ 多走 k 步，之后同步前进， $fast$ 到尾时 $slow$ 位于倒数第 k 个位置。循环条件决定得到的是中点左侧还是右侧，必须在纸上声明“退出时 $fast$ 指向哪里”。
2. **先对齐再同步**：两条链长度不同但共享尾部时，先分别求长度并让长链前移差值，再同步比较地址。比较的是节点身份/地址而不是节点值；值相同不代表共享后继。
3. **局部反转**：回文或重排题可先找中点，再原地反转后半链，最后逐节点比较。反转的不变量是“已处理前缀已经反向，未处理后缀仍保持原链”，循环结束后还要说明是否恢复原链。
4. **快慢相遇**：判环时 $slow$ 每次一步、 $fast$ 每次两步；若存在环，进入环后相对距离每次改变一个模环长，必然相遇。若 $fast$ 或 $fast.next$ 为空，说明链表无环，不能在解引用前跳过边界判断。

这些算法的共同成本不是“指针题都是 $O(n)$ ”，而是一次扫描的基本操作数量、是否需要长度/值域辅助空间、是否允许破坏并恢复原链，以及题目要求返回节点还是返回值。

4.10 数组、矩阵与压缩存储的完整口径

多维数组题至少有三种容易混淆的计数：逻辑下标、每维长度和每个元素占用的存储单元。对 d 维数组 $A[L_1..U_1] \cdots [L_d..U_d]$ ，令 $N_r = U_r - L_r + 1$ ，行优先偏移可写成：

$$offset = \sum_{r=1}^d (i_r - L_r) \prod_{s=r+1}^d N_s, \quad addr = B + offset \cdot w.$$

空维度或越界坐标在进入公式前就已经违反操作契约；公式不能替代合法性检查。列优先只交换“前面完整块”的方向，不应把行优先式中的某一个 N_r 随意换成维度下标。

特殊矩阵压缩的通用步骤是“先判等价类，再给代表元编号”：

1. 对称矩阵：只存上三角或下三角， a_{ij} 与 a_{ji} 先规范化为 $(\max(i, j), \min(i, j))$ 再计算编号。
2. 三角矩阵：按行存储含主对角线的一侧时，第 i 行累计槽位为 $i + 1$ ；带外元素若统一为 c ，需要额外约定一个公共槽位。
3. 对角/三对角矩阵：只保留满足 $|i - j| \leq 1$ 的元素；带内编号随行变化，带外访问直接返回题设常数或零。
4. 稀疏矩阵：三元组按行列记录非零项，或进一步建立行索引；它节省零值空间，却牺牲了任意坐标的直接 $O(1)$ 访问。

压缩不是“少存几个数”这么简单：每个合法逻辑坐标必须映射到唯一独立槽位，每个被省略坐标必须能够由结构规则恢复，否则所谓压缩会丢失信息。反向验证时随机挑选两个对称位置、一个带外位置和一个边界行，检查编号是否越界、是否发生碰撞、是否能还原原值。

4.11 循环链表：只改一条边，所有终止条件都要重写

循环单链表的节点类型与普通单链表相同，区别是尾节点的 *next* 指向头结点（带头结点口径）或首元节点（不带头结点口径），不再指向 *null*。因此必须成套改写：

	普通带头结点链表	循环带头结点链表
判空	$head.next = null$	$head.next = head$
遍历终点	$p = null$	$p = head$
尾节点	$p.next = null$	$p.next = head$

如果循环链表仍使用 $p \neq null$ ，遍历不会自然结束而会绕环；如果普通链表误用 $p \neq head$ ，则到达 *null* 后会非法解引用。插入/删除的局部边操作仍是“先保存原后继，再接入/跨过”，变化只在定位前驱的终止条件。

循环结构还允许只保存尾指针 *rear*：首元节点是 *rear.next.next*（若保留头结点），尾插和头插都可在 $O(1)$ 完成。两个非空循环单链表用尾指针合并时，只需交换两条环在尾部的后继：保存 *A* 的首部，令 *A* 尾接到 *B* 首元，令 *B* 尾接到 *A* 原首；不需要遍历。空表必须先特判，否则可能把待释放的头结点当成数据链使用。

循环双链表把头结点的 *prior* 和 *next* 都指向自己，非空时每个节点满足

$$node.next.prior = node, \quad node.prior.next = node.$$

由于头结点充当首尾两端的哨兵，插入、删除和遍历不再分别特判“没有前驱/没有后继”；只要目标不

是头结点，四条边的局部更新即可保持环。约瑟夫问题则通常选不带头结点的循环单链表：当前报数节点的前驱可直接摘除目标，删除后从其后继继续计数；头结点没有业务含义，反而会干扰报数起点。

4.12 静态链表：把地址换成游标，同时管理两条链

静态链表用数组槽位中的整数下标代替指针地址。约定下标 0 不存数据，只作为备用链表头；数据链另保存一个 *head* 下标，游标 0 表示链尾。于是动态链表的机械对应关系是

$$p \leftrightarrow i, \quad p.next \leftrightarrow space[i].cursor, \quad p = null \leftrightarrow i = 0.$$

静态链表必须同时维护数据链和空闲链：初始化时把所有空闲槽串成 $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 0$ ；申请节点从空闲链头部摘一个，回收节点头插回空闲链。两者都是 $O(1)$ ，但容量固定，按位访问仍需沿游标走 $O(n)$ ，下标也不是逻辑位序。

在已知前驱下插入的游标版仍是

$$space[j].cursor \leftarrow space[k].cursor, \quad space[k].cursor \leftarrow j;$$

删除则先保存 $j = space[k].cursor$ ，令 $space[k].cursor \leftarrow space[j].cursor$ ，再把 j 归还空闲链。若忘记回收，静态数组会逐渐“漏槽”；若先覆盖前驱游标再读取原后继，则和动态链表一样丢链。静态链表不是新的逻辑结构，只是用固定数组承担了动态链表的指针与内存管理责任。

4.13 链表算法的可证明模板

中点：快指针每轮走两步、慢指针走一步，退出条件决定偶数长度取左中点还是右中点；先声明口径，再用它切分或重排。**倒数第 k 个：**前指针先走 k 步，之后与后指针同步，前指针出界等价于后指针距尾 k 个；第一段中必须先检查长度不足，不能在 *null* 上解引用。**判环：**进入环后快慢相对速度为 1，有限模环长下必相遇；找入口时一个指针回到最初出发点，另一个留在相遇点，二者同速相遇于入口。**有序合并：**两条有序链各保留一个游标，每轮摘走更小节点；相等时优先左链才保持稳定，某链耗尽后整体挂接剩余段即可。

链表重排 $a_1, a_2, \dots, a_n \mapsto a_1, a_n, a_2, a_{n-1}, \dots$ 只是三种模板的组合：找中点、保存并断开后半段、原地反转后半段、交替摘接两段。所有就地算法都应在每次覆盖边前保存仍需访问的后继，并最终检查节点数、是否意外成环以及空/单节点/偶数长度边界。

5 同一操作的两套成本向量

不要用一个复杂度数字抹平表示差异。对典型操作，至少记录“是否已知位置/前驱、比较数、移动或跳转数、额外空间和最坏边界”：

操作	顺序表	单链表	成本解释	关键前提
按位访问 i	$\Theta(1)$	$\Theta(i)$, 最坏 $\Theta(n)$	地址计算 vs 从入口逐边跳转	i 合法
按值查找	$\Theta(n)$	$\Theta(n)$	逐项比较	无额外索引
已知插入位序 k	$\Theta(n)$	$\Theta(n)$	顺序表移动后缀; 单链表先定位前驱再改边	只给整数位序
已知前驱后插入	$\Theta(n)$	$\Theta(1)$	顺序表仍需移动; 单链表只改局部边	前驱指针已经给出
头部插入/删除	$\Theta(n)$ / $\Theta(n)$	$\Theta(1)$ / $\Theta(1)$	顺序表整体移动; 链表改入口边	正确维护 head
尾部追加	$\Theta(1)$ 摊还	$\Theta(1)$	几何扩容摊还 vs tail 局部接边	顺序表按固定倍数扩容; 链表维护 tail

空间也要分开：顺序表可能有未使用容量，链表每个节点额外保存指针；两者的逻辑元素空间都是 $\Theta(n)$ ，但常数、局部性和内存管理成本不同。

6 生成性例子：把“按值删除”拆成两个阶段

给定表 $L = [7, 2, 9, 2, 5]$ ，删除第一次出现的 2。这不是一个动作，而是：

1. **Locate**：从头按逻辑次序比较，找到位置 $k = 1$ ；
2. **Update**：顺序表把 $A[2 \dots 4]$ 左移，链表把前驱的后继越过目标节点；
3. **Repair**：更新长度、尾指针、空槽或回收节点；
4. **Verify**：再次遍历，确认次序为 $[7, 9, 2, 5]$ 且所有结构不变量成立。

两种表示的 Locate 都可能是 $\Theta(n)$ ；因此不能看到链表的局部删除就断言“链表删除总是 $O(1)$ ”。真正改变的是 Update 的成本和维护方式。

7 代码把模型变成可执行状态转移

代码不是本册结论之后的语法附录。它必须逐项兑现操作契约、状态字段、不变量修复和边界分支。完整 C++17 实现位于 `code/ds01_linear_list.hpp`；下面的代码块直接从该文件抽取，因此正文与可运行实现共享同一来源。

7.1 顺序表：先保证容量，再从后向前移动

```
1 void insert(std::size_t index, int value) {
2     if (index > length_) {
3         throw std::out_of_range("insert position is outside [0, length]");
4     }
5
6     ensure_capacity(length_ + 1);
7     for (std::size_t i = length_; i > index; --i) {
8         data_[i] = data_[i - 1];
```

```

9      }
10     data_[index] = value;
11     ++length_;
12 }
13
14 int erase(std::size_t index) {
15     require_index(index);
16     const int removed = data_[index];
17     for (std::size_t i = index; i + 1 < length_; ++i) {
18         data_[i] = data_[i + 1];
19     }
20     --length_;
21     return removed;
22 }

```

代码清单 1: 顺序表插入与删除的核心转移

插入循环必须从 *length* 向 *index* 逆向移动；若正向覆盖，尚未复制的旧元素会被破坏。*ensure_capacity* 在移动前执行，保证 $length + 1 \leq capacity$ 。删除则正向覆盖被删位置，并在移动完成后递减长度。两段代码分别把“不覆盖未迁移数据”和“有效区恰为 $[0, length)$ ”落成了执行顺序。

扩容本身是一次显式状态迁移：

```

1  void ensure_capacity(std::size_t required) {
2      if (required <= capacity_) {
3          return;
4      }
5
6      std::size_t new_capacity = capacity_;
7      while (new_capacity < required) {
8          new_capacity *= 2;
9      }
10
11     auto replacement = std::make_unique<int[]>(new_capacity);
12     std::copy_n(data_.get(), length_, replacement.get());
13     data_ = std::move(replacement);
14     capacity_ = new_capacity;
15 }

```

代码清单 2: 容量倍增、复制和所有权切换

旧数据完全复制后才切换 *data_* 与 *capacity_*，因此异常或中途失败不会留下“容量已经变大但数据还没搬完”的半合法状态。一次扩容仍为 $\Theta(n)$ ；只有分析连续追加序列时，倍增策略才给出摊还 $O(1)$ 。

7.2 单链表：局部 $O(1)$ 从“已知前驱”开始

```

1  Node* insert_after(Node* predecessor, int value) {
2      if (predecessor == nullptr) {
3          throw std::invalid_argument("insert_after requires a predecessor");
4      }
5

```

```

6   Node* node = new Node{value, predecessor->next};
7   predecessor->next = node;
8   if (tail_ == predecessor) {
9       tail_ = node;
10  }
11  ++length_;
12  return node;
13  }
14
15  bool erase_after(Node* predecessor) {
16      if (predecessor == nullptr || predecessor->next == nullptr) {
17          return false;
18      }
19
20      Node* removed = predecessor->next;
21      predecessor->next = removed->next;
22      if (tail_ == removed) {
23          tail_ = predecessor;
24      }
25      delete removed;
26      --length_;
27      return true;
28  }

```

代码清单 3: 已知前驱后的插入与删除

插入先令新节点继承旧后继，再让前驱指向新节点；若交换这两步，新节点会把自己接成错误后继或丢失旧链。删除先保存目标节点，跨过它，再修复 `tail_`、释放节点并递减长度。这里的 $O(1)$ 只从传入合法 `predecessor` 开始，不包含寻找前驱。

7.3 按值删除：Locate 与 Update 在代码中也必须分层

```

1   bool erase_first(int value) {
2       Node* predecessor = nullptr;
3       Node* current = head_;
4       while (current != nullptr && current->value != value) {
5           predecessor = current;
6           current = current->next;
7       }
8       if (current == nullptr) {
9           return false;
10      }
11
12      if (predecessor == nullptr) {
13          head_ = current->next;
14          if (tail_ == current) {
15              tail_ = nullptr;
16          }
17          delete current;
18          --length_;
19          return true;

```

```

20     }
21     return erase_after(predecessor);
22 }

```

代码清单 4: 按值删除第一次出现的节点

循环负责 Locate，最坏访问全部 n 个节点；找到后才进入头节点分支或复用 `erase_after`。删除唯一节点时必须同时把 `head_` 和 `tail_` 置空，这正是“边界属于合法状态集合”的代码证据。

7.4 单链表逆置：代码必须先保存旧后继

```

1  void reverse() {
2      Node* old_head = head_;
3      Node* previous = nullptr;
4      Node* current = head_;
5      while (current != nullptr) {
6          Node* next = current->next;
7          current->next = previous;
8          previous = current;
9          current = next;
10     }
11     head_ = previous;
12     tail_ = old_head;
13 }

```

代码清单 5: 单链表原地逆置：保存后继入口后再覆盖当前边

`next` 的存在不是为了“写法好看”，而是保存即将被覆盖的唯一后继入口。逆置完成以后同时交换表头/表尾角色；测试还必须验证新尾节点的 `next` 为空，防止得到一个看似顺序正确、实际仍带旧边或环的结构。

7.5 不变量检查与测试不是生产操作的成本

伴随实现提供 `invariant()`：检查空表字段一致性、环、可达节点数、尾节点身份及“尾节点的 `next` 为空”。它用于教学验证和测试，不意味着每次生产操作都必须额外遍历全表。

最小断言测试位于 `code/ds01_linear_list_test.cpp`，覆盖顺序表扩容与非法位置、链表空表、已知前驱插入、首尾删除、两次逆置恢复和删除至空表。测试的关键部分直接验证抽象结果和结构不变量：

```

1  void test_singly_linked_list() {
2      SinglyLinkedList list;
3      assert(list.empty());
4      assert(list.invariant());
5
6      bool rejected = false;
7      try {
8          list.insert_after(nullptr, 1);
9      } catch (const std::invalid_argument&) {
10         rejected = true;
11     }
12     assert(rejected);

```



```
13
14  list.push_back(7);
15  list.push_back(2);
16  list.push_back(9);
17  auto* predecessor = list.find_first(2);
18  list.insert_after(predecessor, 5);
19  assert((list.values() == std::vector<int>{7, 2, 5, 9}));
20  assert(list.invariant());
21
22  assert(!list.erase_after(list.tail()));
23  assert(list.invariant());
```

代码清单 6: 链表边界与不变量断言

编译验证命令为:

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
2   code/ds01_linear_list_test.cpp -o /tmp/ds01_test
3 /tmp/ds01_test
```

最小调用接口

题面若给位序/下标，先区分“位置到地址能否直接计算”；若只给关键字，定位成本必须单列；只有题目已经给出目标节点或合法前驱时，才进入链式局部改边模型。完整的复杂度辨析、头结点/首元结点、逆置防丢链和插入前提训练见同目录 `.md`。

8 空、首、尾与重复值不是补丁

边界状态	顺序表检查	链式表示检查
空表	$length = 0$ ，不能访问 $A[0]$	$head = null$ ， $tail$ 也必须一致
单元素	删除后长度变为 0	删除后 $head = tail = null$
头部	插入/删除会移动或更新首位置	更新 $head$ ，不能丢掉旧链
尾部	追加可能触发扩容	更新 $tail$ ，保持 $tail.next = null$
重复值	明确删第一次、全部还是指定位置	不能用“找到一个”替代题目契约
越界	先判 $0 \leq k \leq length$	先判指针/位置可达

最常见的错误推理

“链表插入一定比数组快”忽略了定位；“数组访问一定 $O(1)$ ”忽略了按值查找；“长度字段存在就一定正确”忽略了每次分支更新；“删除节点后把指针设空就完成了删除”忽略了前驱、 $head/tail$ 和回收责任。

9 概念边界：把相似说法还原成判据

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
逻辑线性表	≠ 顺序表	前者是关系/操作契约，后者是连续表示	把一种实现当成 ADT 定义
位置已知	≠ 关键字已知	位置可直接进入更新，关键字仍需定位	漏算查找成本
节点已定位	≠ 节点可达	有指针只说明能沿链走，不说明已拿到前驱	把删除写成错误的 $O(1)$
长度	≠ 容量	有效元素数量 vs 已分配槽位数量	扩容/越界条件写反
访问时间	≠ 操作总成本	单次寻址/比较 vs 定位、移动、维护的总和	复杂度结论失真

10 压缩复原

一页压缩

忘掉具体代码后，只需复原五个问题：**对象是什么？输入契约是什么？数据在哪里？操作后必须保持什么？哪一项成本被表示换走了？**连续表示用地址映射换按位访问；链式表示用显式链接换局部插删。所有“快/慢”判断都必须回到操作输入和不变量；完整训练协议由同目录训练 Markdown 拥有。

11 全科坐标与跨册交接

Map Coordinate: Representation / Operation / Invariant / Cost

DS01 把 Atlas 的统一语言落到线性关系：输出“逻辑位置—物理地址/指针—可执行操作—合法状态—成本向量”。DS02 复用顺序/链接表示实现端点限制；DS09 复用有序线性存储讨论查找与索引代价；DS-B01 只把链式/数组状态作为 **frontier** 容器调用。

线性表的本册停止点是“表示如何支撑操作”。一旦题目转而问栈/队列为何产生时间秩序、树/图如何展开关系或索引如何维护有序查询，应交给相应 Owner。

12 来源处理

本册吸收归档《数据结构 MOC》和《算法时间复杂度：统一分析框架》的对象—表示—操作—成本语言，并将算法笔记中的列表/队列/字典 API 仅作为实现例子，不把 Python 容器行为提升为 408 数据结构定义。扩容摊还、缓存局部性和泛型容器保留为 Extension；栈、队列、串、树、图、Hash 与排序分别由其他 Topic Own。

本轮基础概念复审另外核对了以下公开教学材料：MIT 6.005 的 ADT / Representation Independence 用于确认“抽象操作契约与具体表示分离”的边界；OpenDSA 的 List ADT、array-based list、linked list 与 implementation comparison 用于核对按位访问、插删和“已知位置”前提；CMU linked-list 教学材料用于核对链表入口、指针可达性与逆置时的丢链风险。这些材料只用于验证共同机制，不替代 408 的术语口径。

- MIT 6.005, *Abstract Data Types*: <https://ocw.mit.edu/ans7870/6/6.005/s16/classes/12-abstract-data-types/index.html>

- OpenDSA, *Comparison of List Implementations*: https://opensa-server.cs.vt.edu/ODSA/Books/winthrop/csci271/fall-2023/TRF_930am/html/ListAnalysis.html
- OpenDSA, *Linked Lists*: <https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/ListLinked.html>
- Carnegie Mellon University, *Linked Lists – Introduction / Advanced Operations*: https://www.cs.cmu.edu/~clo/www/CMU/DataStructures/Lessons/lesson1_2.htm